Manish Khatiwada

Nov 09, 2026

Assignment 05

Advanced Collections and Error Handling

Introduction

In this fifth assignment and Module, we learned about

a. **Using Dictionaries with files:** A dictionary stores data as key-value pairs. We can think of it as one row in a spreadsheet, where each column name is a key and the cell beneath it is the value. we retrieve any value by its key, such as row. Keys are case-sensitive. Just as with a list, we can fill a dictionary with user input or with data read from a text file. A list stores items in an ordered sequence, and we access them by position whereas dictionary stores key-value pairs and you access values by their key.

b. **Using JSON Files:** JSON (JavaScript Object Notation) is a lightweight text format for storing and exchanging data. We can read and write it easily, and programs can parse and generate it quickly. It is commonly used for web APIs, client-server communication, configuration files, and document-oriented databases. JSON data is organized as key-value pairs, where each key is a string in double quotes, and values can be nested to represent simple or complex structures. To write a Python dictionary or list to a JSON file, we can convert it to a string manually, but the built-in json module does this.

c. **Try/Except:** Error handling lets a script deal with problems it cannot otherwise prevent, such as a missing file or invalid user input. Instead of crashing, the script can show a clear, user-friendly message, keep its error responses in one organized place, and jump to a separate block of code that you write to decide how each error is handled. When an error occurs, Python automatically creates an Exception object from the Exception class and fills it with details about what went wrong. The base Exception class catches any type of error, but we can catch specific problems with more specific classes, such as ValueError or FileNotFoundError. In a try-except block, you can capture the Exception object with as and read its message.

The optional finally block runs after the try and except blocks finish, whether an error occurred.

d. **Using The GitHub Website:** GitHub is a cloud-based platform for code hosting and collaboration rather than a traditional file-sharing service. It does include file-sharing elements, but its core purpose is version control for software projects. Developers and teams use it to share, manage, and collaborate on code and related files in one place, which makes it a powerful tool for building software together. In this module, we created a GitHub account and then a repository, which we set as either public or private. After creating the repository, we uploaded files to it.

Creating a Program (Assignment05.py)

1. **Header comment block:** The script begins with a header comment block. Every line starting with # is a comment and is ignored by Python. The block gives the title and description of the script so that other people reviewing it understand its intent. Assignment 05 focused on use of dictionaries, json file, and exception handling.

```
# ------------------------------------------------------------------------
------------------ #
# Title: Assignment05
# Desc: This assignment demonstrates using dictionaries, files, and
exception handling
# Change Log: (Who, When, What)
#   RRoot,1/1/2030,Created Script
#   Manish Khatiwada, 11/09/2026, Created Script
# ------------------------------------------------------------------------
------------------ #
```

2. **Importing json and JSONDecodeError:** import json loads the python built json module so we can read and write json file whereas JSONDecodeError loads the error Python raises when a file contains invalid json.

```
# Import the json
import json
from json import JSONDecodeError
```

3. **Creating Constant (Setup code):** I created two constants named MENU type string that stores course registration program information, and FILE_NAME type string that stores “Enrollments.json”. It is written within the constant section so if we need to rename the file, we could do just at this place instead of looking throughout the code. Constants are written in all caps by convention to signal that the value should not change while the script runs.

```
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------
'''
FILE_NAME: str = "Enrollments.json"
```

4. **Creating Variables (Setup code):** I created seven variables, student_first_name with empty string, student_last_name with empty string, course_name with empty string, menu choice that will start with empty string, student_data is set to empty dict, students is set to empty list, and file set to None.

```
# Define the Data Variables and constants
student_first_name: str = ''  # Holds the first name of a student entered
by the user
student_last_name: str = ''  # Holds the last name of a student entered by
the user
course_name: str = ''  # Holds the name of a course entered by the user
student_data: dict = {}  # one row of student data
students: list = []  # a table of student data
file = None  # Holds a reference to an opened file
menu_choice: str = '' # Hold the choice made by the user
```

5. **Creating Input (Main body of a script):**

   The first step of the script is to read the file data into a list of dictionaries. It loads saved student data from a JSON file when the program starts and handles errors. The try block opens the file in read mode, json.load converts the JSON into Python data stored in students as a list of dictionaries, and the file is closed. If the file does not exist yet, the FileNotFoundError except block prints a message with technical details, creates the file in write mode, and writes the empty students list to it. The

finally block runs whether or not an error occurred and closes the file if it is still open.

```
# When the program starts, read the file data into a list of
dictionaries(table)
# Extract the data from the file
try:
    file = open(FILE_NAME, "r")
    students = json.load(file)
    file.close()
except FileNotFoundError as e:
    print(f"File {FILE_NAME} not found")
    print('-----Technical Error------')
    print(e, e.__doc__, type(e), sep='\n')
    print("creating non-existent file")
    file = open(FILE_NAME, "w")
    json.dump(students, file)
    file.close()
except JSONDecodeError as e:
    print(f"File {FILE_NAME} is not valid JSON")
    print('-----Technical Error------')
    print(e, e.__doc__, type(e), sep='\n')
except Exception as e:
    print("There was a non-specific error!")
    print('-----Technical Error------')
    print(e, e.__doc__, type(e), sep='\n')
finally:
    # Check if a file object exists and is still open
    if file is not None and file.closed == False:
        file.close()
```

In the second step, the main loop of the program starts using a while loop. It shows the menu and asks for a choice until the user exits.

```
# Present and Process the data
while (True):

    # Present the menu of choices
    print(MENU)
    menu_choice = input("What would you like to do: ")
```

In the third step, if/elif handles one menu option at a time, and the loop repeats until option 4 breaks out. I started with option 1 and registered student Rigdha Acharya to the Python 201 course. The script checks the first name and the last name with isalpha(). If either is not made up of letters, a ValueError is raised. The except ValueError block prints a message, and continue sends the user back to the menu. Finally, the generic except Exception block catches all unexpected errors.

```
# Input user data
if menu_choice == "1":  # This will not work if it is an integer!
```

```
    try:
        student_first_name = input("Enter the student's first name: ")
        if not student_first_name.isalpha():
            raise ValueError("The first name should not contain numbers.")
    except ValueError as e:
        print(e)
        print('-----Technical Error------')
        print(e.__doc__)
        continue
    except Exception as e:
        print("There was a non-specific error!")
        print(e, e.__doc__, type(e), sep='\n')
        continue
    try:
        student_last_name = input("Enter the student's last name: ")
        if not student_last_name.isalpha():
            raise ValueError("The last name should not contain numbers.")
    except ValueError as e:
        print(e)
        print('-----Technical Error------')
        print(e.__doc__)
        continue
    except Exception as e:
        print("There was a non-specific error!")
        print(e, e.__doc__, type(e), sep='\n')
        continue

    course_name = input("Please enter the name of the course: ")
    student_data = {'FirstName': student_first_name,
                    'LastName': student_last_name,
                    'CourseName': course_name}
    students.append(student_data)
    print(f"You have registered {student_first_name} {student_last_name} for
{course_name}.")
    continue
```

6. **Output (Main body of a script):**

In the fourth step, I selected option 2, which shows the current data. It prints each student as a readable sentence and then each student as comma-separated values. I selected option 3 to save the file. It opens the file in write mode, uses json.dump to write the students list, closes the file, and prints what was saved. If a TypeError occurs, the data cannot be converted to JSON and an error message is shown. I finally selected option 4 to exit, which breaks the while loop and ends the program.

```
# Present the current data
    elif menu_choice == "2":

        # Process the data to create and display a custom message
        print("-"*50)
```

```
        for student in students:
            print(f"Student {student['FirstName']} {student['LastName']} is
enrolled in {student['CourseName']}")
        print("-"*50)

        # Show a string of comma-separated values for each row
        for student in students:

print(f"{student['FirstName']},{student['LastName']},{student['CourseName']}"
)
        print("-" * 50)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        try:
            file = open(FILE_NAME, "w")
            json.dump(students, file)
            file.close()
            print("The following data was saved to file")
            for student in students:
                print(f"Student {student['FirstName']} {student['LastName']}
is enrolled in {student['CourseName']}")
        except TypeError as e:
            print("Please check that the data is a valid JSON format")
            print('-----Technical Error------')
            print(e, e.__doc__, type(e), sep='\n')
        except Exception as e:
            print('-----Technical Error------')
            print(e, e.__doc__, type(e), sep='\n')
        finally:
            # Check if a file object exists and is still open
            if file is not None and file.closed == False:
                file.close()
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop
    else:
        print("Please only choose option 1, 2, 3, or 4")

print("Program Ended")
```

7. Execution of the script: I saved Assignment05.py in PyCharm and ran it using Run Assignment05.py. I also ran Assignment05.py in the macOS terminal (command shell). Output is displayed below.
   Note: In PyCharm, I registered Rigdha Acharya for Python 201 using menu option 1. I then added Manish Khatiwada for Python 202 directly to the Enrollments.json file as additional data. In the macOS terminal, I registered Kanti Acharya for 203 using menu option 1. See the screenshot of the JSON file (Enrollments.json) for details.

Python Code created in PyCharm

```
# ------------------------------------------------------------------------
-------------- #
# Title: Assignment05
# Desc: This assignment demonstrates using dictionaries, files, and exception
handling
# Change Log: (Who, When, What)
#   RRoot,1/1/2030,Created Script
#   Manish Khatiwada, 11/09/2026, Created Script
# ------------------------------------------------------------------------
-------------- #

# Import the json
import json
from json import JSONDecodeError

# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course
    2. Show current data
    3. Save data to a file
    4. Exit the program
-----------------------------------------
'''
# Define the Data Constants
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables and constants
student_first_name: str = ''  # Holds the first name of a student entered by
the user
student_last_name: str = ''  # Holds the last name of a student entered by
the user
course_name: str = ''  # Holds the name of a course entered by the user
student_data: dict = {}  # one row of student data
students: list = []  # a table of student data
file = None  # Holds a reference to an opened file
menu_choice: str = '' # Hold the choice made by the user


# When the program starts, read the file data into a list of
dictionaries(table)
# Extract the data from the file
try:
    file = open(FILE_NAME, "r")
    students = json.load(file)
    file.close()
except FileNotFoundError as e:
    print(f"File {FILE_NAME} not found")
    print('-----Technical Error------')
    print(e, e.__doc__, type(e), sep='\n')
    print("creating non-existent file")
    file = open(FILE_NAME, "w")
    json.dump(students, file)
```

```
    file.close()
except JSONDecodeError as e:
    print(f"File {FILE_NAME} is not valid JSON")
    print('-----Technical Error------')
    print(e, e.__doc__, type(e), sep='\n')
except Exception as e:
    print("There was a non-specific error!")
    print('-----Technical Error------')
    print(e, e.__doc__, type(e), sep='\n')
finally:
    # Check if a file object exists and is still open
    if file is not None and file.closed == False:
        file.close()

# Present and Process the data
while (True):

    # Present the menu of choices
    print(MENU)
    menu_choice = input("What would you like to do: ")

    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        try:
            student_first_name = input("Enter the student's first name: ")
            if not student_first_name.isalpha():
                raise ValueError("The first name should not contain
numbers.")
        except ValueError as e:
            print(e)
            print('-----Technical Error------')
            print(e.__doc__)
            continue
        except Exception as e:
            print("There was a non-specific error!")
            print(e, e.__doc__, type(e), sep='\n')
            continue
        try:
            student_last_name = input("Enter the student's last name: ")
            if not student_last_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
        except ValueError as e:
            print(e)
            print('-----Technical Error------')
            print(e.__doc__)
            continue
        except Exception as e:
            print("There was a non-specific error!")
            print(e, e.__doc__, type(e), sep='\n')
            continue

        course_name = input("Please enter the name of the course: ")
        student_data = {'FirstName': student_first_name,
                        'LastName': student_last_name,
                        'CourseName': course_name}
        students.append(student_data)
        print(f"You have registered {student_first_name} {student_last_name}
```

```
for {course_name}.")
        continue

    # Present the current data
    elif menu_choice == "2":

        # Process the data to create and display a custom message
        print("-"*50)
        for student in students:
            print(f"Student {student['FirstName']} {student['LastName']} is
enrolled in {student['CourseName']}")
        print("-"*50)

        # Show a string of comma-separated values for each row
        for student in students:

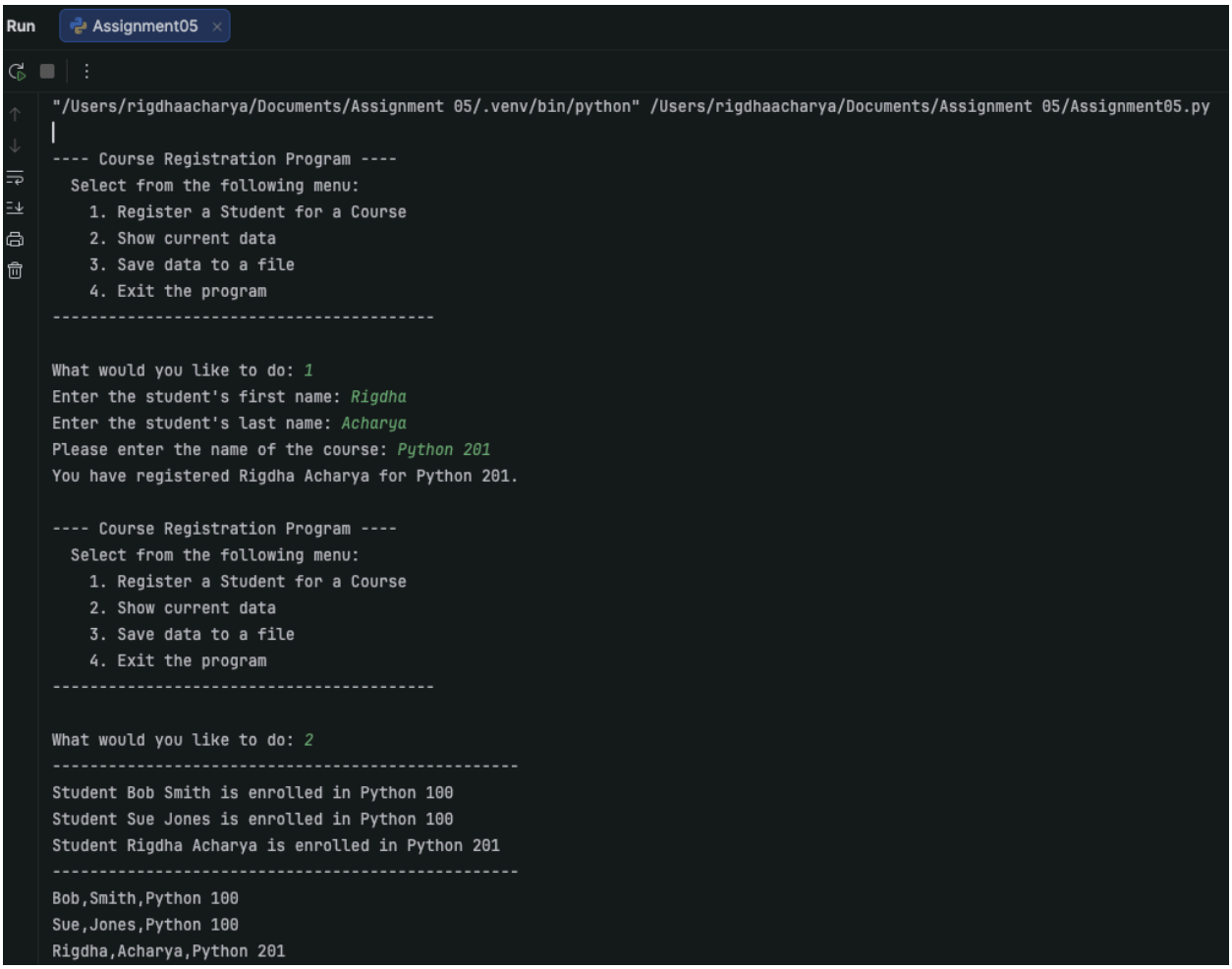
print(f"{student['FirstName']},{student['LastName']},{student['CourseName']}"
)
        print("-" * 50)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        try:
            file = open(FILE_NAME, "w")
            json.dump(students, file)
            file.close()
            print("The following data was saved to file")
            for student in students:
                print(f"Student {student['FirstName']} {student['LastName']}
is enrolled in {student['CourseName']}")
        except TypeError as e:
            print("Please check that the data is a valid JSON format")
            print('-----Technical Error------')
            print(e, e.__doc__, type(e), sep='\n')
        except Exception as e:
            print('-----Technical Error------')
            print(e, e.__doc__, type(e), sep='\n')
        finally:
            # Check if a file object exists and is still open
            if file is not None and file.closed == False:
                file.close()
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop
    else:
        print("Please only choose option 1, 2, 3, or 4")

print("Program Ended")
```

Output when we running python in PyCharm


Run
Assignment05
"/Users/rigdhaacharya/Documents/Assignment 05/.venv/bin/python" /Users/rigdhaacharya/Documents/Assignment 05/Assignment05.py
---- Course Registration Program ----
Select from the following menu:
1. Register a Student for a Course
2. Show current data
3. Save data to a file
4. Exit the program
What would you like to do: 1
Enter the student's first name: Rigdha
Enter the student's last name: Acharya
Please enter the name of the course: Python 201
You have registered Rigdha Acharya for Python 201.
---- Course Registration Program ----
Select from the following menu:
1. Register a Student for a Course
2. Show current data
3. Save data to a file
4. Exit the program
What would you like to do: 2
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Rigdha Acharya is enrolled in Python 201
Bob,Smith,Python 100
Sue,Jones,Python 100
Rigdha,Acharya,Python 201

```
----------------------------------------------

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------

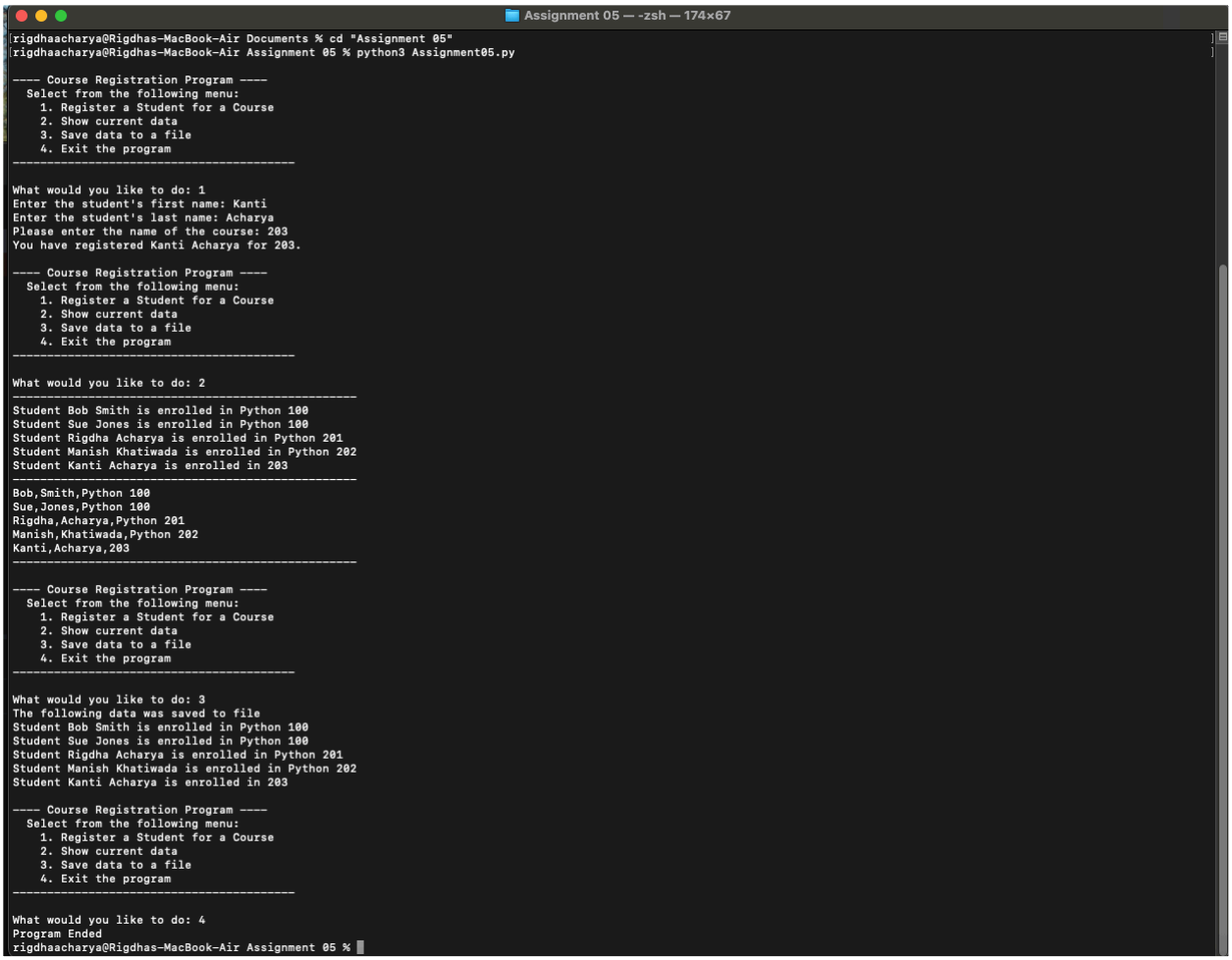
What would you like to do: 3
The following data was saved to file
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Rigdha Acharya is enrolled in Python 201

---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------

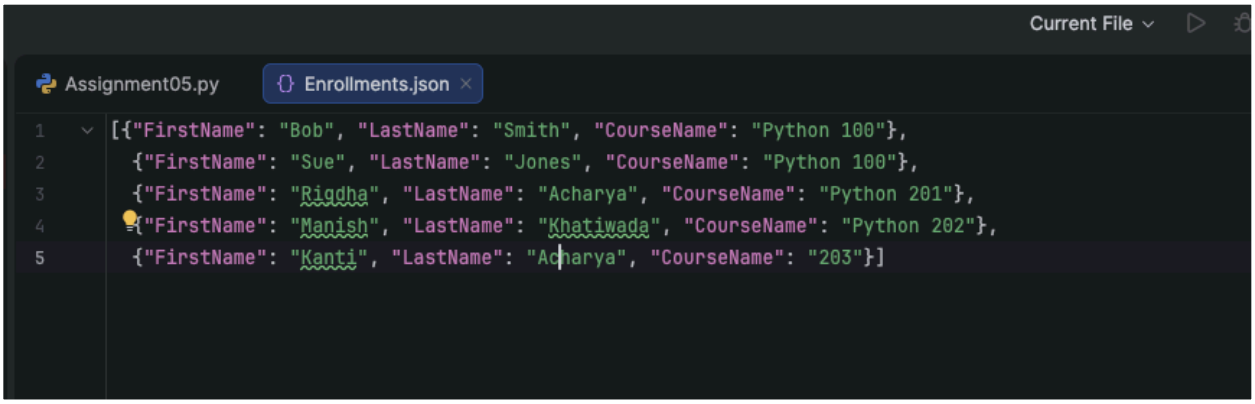
What would you like to do: 4
Program Ended

Process finished with exit code 0
```

Running the script from the terminal (command shell) on macOS. The Assignment 05 folder was created in the Documents folder, where Assignment05.py is stored.



Screenshot of the json file



Summary or Conclusion: In this module and assignment, I created a course registration program that stores each registration as a dictionary and keeps all registrations in a list of

dictionaries. The program loads saved data from Enrollments.json when it starts, and the json module converts between the file and Python data. In addition, I learned about structured error handling with try/except. The program shows a clear message and returns to the menu. The finally block makes sure the file is closed whether or not an error occurs. In PyCharm and in the macOS terminal, I entered multiple registrations, displayed them, saved them to Enrollments.json, and exited. In addition, I posted the script and this document to a GitHub repository, which allow to share, manage, and collaborate on code and related files in one place.